

Relatório de Incidente

Cardápio Digital indisponível — cardapio.bodogami.tech

Data do incidente	01-02/09/2026
Sistema afetado	Cardápio Digital (VPS) — cardapio.bodogami.tech
Sintoma reportado	"Não foi possível carregar o cardápio. Verifique sua conexão."
Status final	Resolvido — cardápio carregando normalmente
Commit da correção	20700e5 — branch firebase

Resumo Executivo

O cardápio digital público, hospedado na VPS e acessado pelos clientes do restaurante, parou de carregar, exibindo uma mensagem genérica de erro de conexão. A investigação revelou **seis problemas independentes e empilhados** na cadeia de comunicação entre a VPS e o backend local do restaurante (via túnel WireGuard), acumulados ao longo de semanas sem terem sido detectados — o sistema nunca havia sido testado de ponta a ponta desde que essa integração foi implantada.

Nenhum dado de clientes ou do restaurante foi perdido. Uma migração de banco de dados que estava pendente (planejada, mas nunca finalizada) foi identificada e concluída durante a correção, incluindo validação de integridade dos dados migrados.

Causas Raiz (na ordem em que foram descobertas)

1 URL do backend local apontando para porta errada

LOCAL_BACKEND_URL no .env.vps da VPS apontava para a porta 3000 ; o backend real escuta na 3001 .

Correção: valor ajustado para http://10.0.0.2:3001 .

2 Porta do backend não publicada no host de produção

O serviço backend no docker-compose.prod.yml não expunha nenhuma porta ao host Windows — só era acessível pela rede interna do Docker, tornando-o inalcançável pelo túnel WireGuard.

Correção: publicada em 127.0.0.1:3001 (loopback apenas, sem expor à LAN/internet).

3 Migração de banco de dados pendente e nunca finalizada

Ao recriar o container do backend para aplicar a correção #2, o processo entrou em **crash-loop**: o `.env` de produção já apontava para um Postgres externo (`10.5.103.73`), mas os dados reais (cardápio, usuários, comandas) ainda estavam apenas no Postgres interno (container `bodogami_prod_postgres`, no ar desde 18/08). O backend anterior só funcionava porque nunca havia sido recriado desde a mudança do `.env`.

Correção: dump completo do Postgres interno (`pg_dump`), restauração no externo (`pg_restore`), e conferência de integridade por contagem de linhas em todas as tabelas principais — 100% de correspondência. Backend reapontado para o banco externo.

4 Porta em loopback não alcançável pelo túnel WireGuard

O WireGuard roda dentro de uma distro WSL Ubuntu, numa interface de rede própria (`10.0.0.2`) — distinta do mecanismo de "localhost forwarding" do WSL2 que só cobre `127.0.0.1`. Tráfego chegando em `10.0.0.2:3001` não tinha para onde ir.

Correção: encaminhamento com `socat`, vinculado especificamente à interface do WireGuard, redirecionando para `127.0.0.1:3001`. Uma tentativa inicial com `iptables REDIRECT` não funcionou de forma confiável (problema de conntrack/roteamento assimétrico com a interface de túnel). Serviço tornado persistente via `systemd` (`cardapio-vpn-forward.service`), sobrevivendo a reinicializações da WSL.

5 Healthcheck do vps-api sempre falhando (bug de IPv6)

O healthcheck usava `http://localhost:4000/health`. Em imagens Alpine (musl libc), `localhost` pode resolver para `::1` (IPv6), endereço que o Node não escuta — o container ficava permanentemente marcado `unhealthy`, mesmo com o servidor respondendo normalmente.

Correção: healthcheck alterado para `http://127.0.0.1:4000/health` em todos os arquivos que o definem.

6 Causa raiz definitiva — label do Traefik ausente desde a criação do arquivo

O container `vps-api` define **dois** services Traefik na mesma porta (`okaikiei-api` e `okaikiei-internal`). Quando um container define mais de um service, o Traefik exige o label explícito `traefik.http.routers.<nome>.service=<nome>` para saber qual router vai com qual service — sem ele, os dois routers são **descartados silenciosamente**, sem gerar erro visível em uso normal (só apareceu com log em nível DEBUG). Esse bug existia desde a criação do arquivo; nenhuma das correções anteriores (#1–#5) resolveria o problema sozinha por causa dele.

Correção: adicionados os labels `traefik.http.routers.okaikiei-api.service=okaikiei-api` e `traefik.http.routers.okaikiei-internal.service=okaikiei-internal`.

Método de Diagnóstico

Cada camada da cadeia foi testada isoladamente até isolar exatamente onde a requisição parava:

Camada	Teste	Resultado antes	Resultado depois
Backend → Postgres	<code>docker logs</code> do backend	Crash-loop (auth falhou)	Conectado
WSL → Windows (loopback)	<code>curl localhost:3001</code>	OK (mascarava o problema real)	OK
WireGuard → Backend	<code>curl 10.0.0.2:3001</code> (de dentro da WSL e da VPS)	Recusado	200/401 corretos
vps-api → Backend local	<code>docker logs okaikei_vps_api</code>	ECONNREFUSED	"Menu cached"
Docker healthcheck	<code>docker ps</code>	unhealthy	healthy
Traefik → vps-api	log DEBUG do Traefik	Router descartado (erro de linking)	Router ativo
Ponta a ponta (público)	<code>curl https://cardapio.bodogami.tech/api/menu</code>	HTML do frontend (fallback)	JSON real do cardápio

Alterações no Repositório (versionadas)

Commit `20700e5` na branch `firebase` :

- `.github/workflows/deploy-vps.yml` — healthcheck do CI/CD corrigido (IPv6)
- `docker-compose.hostinger.yml` — labels `.service=` do Traefik + healthcheck
- `docker-compose.prod.yml` — porta publicada + `DATABASE_URL` apontando pro banco externo
- `docker-compose.vps.yml` — healthcheck corrigido (IPv6)

Alterações Fora do Controle de Versão

Aplicadas diretamente na infraestrutura, **não** versionadas em git — importante para continuidade caso a equipe precise reconstruir o ambiente do zero:

- Servidor do restaurante (Windows/WSL):** serviço systemd `cardapio-vpn-forward.service` criado em `/etc/systemd/system/` na distro WSL Ubuntu, executando `socat TCP-LISTEN:3001,bind=10.0.0.2,fork,reuseaddr TCP:127.0.0.1:3001` , habilitado para iniciar no boot.
- Postgres externo (10.5.103.73):** regra de `pg_hba.conf` liberando a conexão do host Windows/Docker Desktop do restaurante (ajustada pela equipe durante o incidente).
- Migração de dados:** dump/restore completo do banco `bodogami_prod` (interno) para `okaikei` (externo, 10.5.103.73), incluindo schema, dados, sequences, constraints e índices.

Recomendações

- **Documentar o serviço systemd** do encaminhamento WireGuard→Docker no repositório (ex: `vps-api/CLAUDE.md` ou um novo `docs/`), já que hoje só existe na memória de quem configurou — se a WSL for reinstalada, essa etapa será perdida.
- **Testar o fluxo ponta a ponta após qualquer deploy** que toque backend, vps-api, ou configuração do Traefik — nenhum desses seis problemas gerava erro óbvio isoladamente; só o teste real do `https://cardapio.bodogami.tech/api/menu` expõe a cadeia completa.
- **Decidir o destino do Postgres interno** (`bodogami_prod_postgres`) — ainda está no ar, sem uso, desde a migração. Pode ser mantido como backup temporário por um período de segurança e depois desligado.
- **Investigar o aviso de NATS** ("Authorization Violation") observado nos logs do backend durante o incidente — não bloqueia a aplicação, mas desativa eventos locais; não foi resolvido nesta investigação.
- **Considerar adicionar um healthcheck de ponta a ponta** (ex: um cron simples testando a URL pública e alertando em caso de falha) para detectar esse tipo de regressão automaticamente, já que o problema ficou sem ser notado por semanas.